Plano de Verificação

Sistema `riscv-dma-matrix-coprocessor`

CPU RV32I, controlador de DMA, acelerador sistólico *weight-stationary* parametrizável ($N = 2, 4, 8$) e memória compartilhada

DUTs: `accel_systolic` · `accel_top` · CSR · DMA · memória compartilhada · barramento AXI4-Lite · `soc_top`

Gustavo Santiago, Leandro Gama, Valerio Gabriel, Carolina Correia

Equipe do projeto `riscv-dma-matrix-coprocessor`

CEPEDI — CI Digital

Programa de Desenvolvimento de Competências em Sistemas Digitais

Documento	Plano de Verificação (versão inicial)
Versão	0.1
Data	14/11/2026
Base	READMEs de `02.Architecture`, `03.Reference`, `04.RTL` e `05.Verification`; RTL do acelerador (`accel_top.sv`, `accel_systolic.sv`) e `axil_if.sv`; enunciado do projeto e enunciado do trabalho final de verificação; commit `f8fc43c` de 14/11/2026

Sumário

1 Introdução **2**
1.1 Visão geral do projeto . . . 2
1.2 Descrição dos DUTs . . . 2
1.3 Objetivos e metas da verificação . . . 5
1.4 Ambiente e ferramentas . . . 5

2 Plano de teste **8**
2.1 Features a serem verificadas . . . 8
2.1.1 Pontos de atenção levantados na revisão do RTL . . . 11
2.2 Estratégias de inserção de estímulos . . . 13
2.3 Modelo de cobertura . . . 14
2.4 Resumo dos testes planejados . . . 14

3 Cronograma preliminar **16**

Histórico de revisões

Versão	Data	Descrição
0.1	14/11/2026	Versão inicial: introdução, DUTs, metas, plano de teste, cobertura e cronograma preliminar. Base: RTL do acelerador (E2), golden model e ICD parcial.

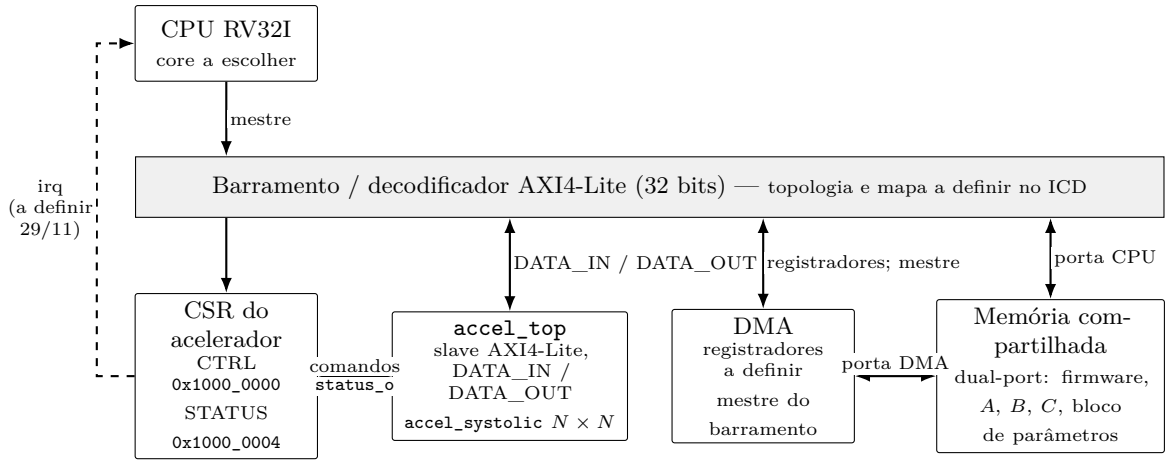


Figura 1: Sistema a verificar. Os comandos do CSR para o acelerador são `mode_i`, `load_i`, `start_i` e `clr_error_i`; o acelerador devolve `status_o`. Só o acelerador, o CSR (como contrato) e a interface `axil_if` existem em 14/11/2026; os demais blocos seguem o enunciado e o roadmap.

1 Introdução

1.1 Visão geral do projeto

O projeto é um sistema digital que calcula $C = A \times B$. Uma CPU RISC-V RV32I apenas configura endereços e dimensões e dispara a operação; um controlador de DMA move, de forma autônoma, os blocos de A e B da memória compartilhada para o buffer de entrada do acelerador, emite os comandos de carga e de execução e grava os blocos de C de volta na memória; o acelerador é um *systolic array weight-stationary* $N \times N$ ($N \in \{2, 4, 8\}$) com elementos de `DATA_WIDTH` $\in \{8, 16, 32\}$ bits, acumulador interno de $2 \cdot DW + \lceil \log_2 N \rceil$ bits e saída de DW bits com *wrap* (sem saturação). Matrizes maiores que a malha são particionadas em blocos (*tiling*) fora do acelerador. O tráfego de controle usa AXI4-Lite de 32 bits. O término é sinalizado por registrador de status; a adoção de interrupção será decidida em 29/11/2026. O enunciado exige a síntese de pelo menos três configurações da malha e um ambiente UVM integrado que compare automaticamente a matriz C produzida pelo hardware com um modelo de referência.

A verificação se apoia em três fontes de referência:

1. o **golden model em Python** (`03.Reference/golden/systolic_array.py`), modelo funcional da multiplicação por blocos, com a mesma máscara de acumulador e o mesmo *wrap* do RTL;
2. os **vetores gerados** por `generate_rtl_vectors.py`: 81 casos (9 configurações $N \times DW$ vezes 9 casos), com seed 2026, em arquivos texto que o testbench lê;
3. **modelos de transação em SystemVerilog** para o barramento AXI4-Lite, os registradores (CSR e DMA) e a sequência de comandos do acelerador.

Não há co-simulação instrução a instrução do core: ele é verificado pelo seu dono com programas autoverificáveis e, neste plano, é tratado como o executor do firmware.

1.2 Descrição dos DUTs

A Figura 1 mostra o sistema e a Tabela 1 os três níveis em que ele é verificado. Cada bloco entra no subsistema e no sistema só depois de passar no seu próprio ambiente.

Acelerador. A casca `accel_top` tem um slave AXI4-Lite de 32 bits (`axil_if.target`), um buffer de entrada `input_q` de `BUFFER_WORDS` $= N^2 \cdot DW/32$ palavras e um buffer de saída `output_words`, que é um apelido dos registradores `result_q` do array. Admite uma escrita e uma leitura pendentes; `bvalid` sobe um ciclo após o último handshake de AW/W e `rvalid` um ciclo após o de AR; `wstrb` habilita bytes; respondem SLVERR o endereço desalinhado, o endereço fora da faixa, a escrita fora de IDLE, a escrita em DATA_OUT e a leitura de DATA_IN; `awprot`/`arprot` são ignorados e os erros AXI não alteram o bit ERROR. O núcleo `accel_systolic` tem a FSM IDLE (`00`) / LOADING (`01`) / PROCESSING (`10`): LOAD só é aceito em IDLE com `mode_i`=0 e START só em IDLE com `mode_i`=1, nunca os dois

Tabela 1: Níveis de verificação, DUTs e ambientes.

Nível	DUTs	Ambiente
N1 Unidade	Acelerador: `accel_systolic` e `accel_top` (slave AXI4-Lite, buffers e array); golden model; memória compartilhada; CSR do acelerador; DMA isolado (com memória e acelerador comportamentais); core RV32I isolado.	Testbenches SystemVerilog dirigidos com tasks e linha `RESUMO: n OK, m ERRO` (`05.Verification/unit/`); o do acelerador já existe (`accel_top_tb.sv`: 9 instâncias $N \times DW$, 81 vetores do golden); `unittest` Python (`make test`); programas autoverificáveis do core contra memória-modelo.
N2 Bloco e subsistema	Acelerador com agente AXI4-Lite UVM (o ambiente faz o papel do DMA, três valores de N); **subsistema E3**: DMA + memória + CSR + acelerador, sem CPU (o testbench faz o papel da CPU).	UVM em `05.Verification/subsys/`: agente `op` (sequencer, driver A = BFM AXI4-Lite que programa DMA e CSR, monitor), agente AXI4-Lite passivo, modelo de referência, scoreboard e cobertura; SVA em arquivos de `bind`.
N3 Sistema	`soc_top`: CPU + barramento/decodificador + CSR + DMA + memória compartilhada + acelerador, executando o firmware.	UVM integrado em `05.Verification/soc/`, reaproveitando o ambiente N2: driver B (carga das memórias por *backdoor* e firmware genérico) por *factory override*; monitor lê C na memória; agentes de barramento passivos; vetores do golden em arquivo e modelo de referência por TLM.

Tabela 2: Mapa de endereços: o que está definido e o que falta.

Base	Tamanho	Região	CPU	DMA	Fonte / situação
a definir	a definir	Memória de instruções (firmware `.hex`) e vetor de reset	busca	—	ICD (a definir)
a definir	a definir	Memória de dados compartilhada: A, B, C e bloco de parâmetros	`lw`/`sw`	lê A e B, escreve C	ICD (a definir); dual-port (`04.RTL/soc`)
`0x0000_0000`	até `0x4000_0000`	"Faixa de CSR" declarada em `csr_readme.md`	—	—	**inconsistente**: sobrepõe as memórias e deixa os buffers de fora (P10)
`0x1000_0000`	4 B	Acelerador CTRL: MODE (bit 0, RW); LOAD, START, CLR_ERROR (bits 1–3, W1P)	`sw`/`lw`	a definir (P9)	`csr_readme.md`; RTL não existe
`0x1000_0004`	4 B	Acelerador STATUS (RO) = `status_o`: STATE[1:0], DONE[2], ERROR[3]	`lw`	a definir	idem
`0x1000_0008` a `0x2000_0000`	—	Reservado ao acelerador	—	—	`csr_readme.md`
a definir	a definir	Registradores do DMA: bases de A, B, C; dimensões; CTRL; STATUS	`sw`/`lw`	—	`csr_readme.md`: "to be defined"
`0x4000_1000`	$N^2 \cdot DW/8$ B (4 a 256)	`accel_top` DATA_IN (só escrita; OKAY só em IDLE)	—	escrita	parâmetro `DATA_IN_BASE`
`0x4000_2000`	$N^2 \cdot DW/8$ B	`accel_top` DATA_OUT (só leitura; aceita em qualquer estado)	—	leitura	parâmetro `DATA_OUT_BASE`
qualquer outro	—	Resposta a definir (proposta: SLVERR/DECERR no decodificador)			ICD (a definir)

juntos; qualquer outro LOAD/START liga ERROR (`status_o[3]`), pegajoso até `clr_error_i`; DONE (`status_o[2]`) é limpo na aceitação e ligado ao fim. LOADING dura N ciclos (a linha r de B entra no ciclo r); PROCESSING dura $3N-2$ ciclos e $C[i][j]$ é capturado no ciclo $i+N-1+j$; o acumulador tem $2 \cdot DW + \lceil \log_2 N \rceil$ bits e a saída guarda os DW bits baixos. Os pesos persistem entre STARTs. Não há interrupção.

CSR do acelerador. Contrato em `02.Architecture/icd/csr_readme.md`: CTRL e STATUS de 32 bits; MODE retido; LOAD, START e CLR_ERROR geram um pulso de um ciclo por transação (W1P). O RTL não existe e o dono do bloco não está definido.

DMA. Requisitos da seção 5 do enunciado: registradores de endereço-base, dimensões, controle e status; FSM de leitura de A e B, escrita de C e contadores de endereço; operação independente da CPU após a configuração; coordenação com o acelerador por sinais de requisição, pronto/ocupado e conclusão. O RTL está previsto para 17–18/11 e os registradores estão "a definir".

Memória compartilhada e barramento. Memória dual-port (porta da CPU e porta do DMA) e decodificador/interconexão AXI4-Lite. Só existe a interface genérica `axil_if.sv` (sinais AW, W, B, AR, R e modports `initiator`/`target`).

CPU e firmware. Core RV32I (escolha pendente, por ADR: próprio ou de terceiros) e firmware *freestanding* que lê um bloco de parâmetros, programa DMA e CSR por ponteiros `volatile` de um header gerado do ICD, dispara a operação e espera o término. O `.hex` é versionado em `06.Firmware/`.

Golden model. A função `systolic_array` recebe `a`, `b`, `array_size`, `data_width` e `signed`; multiplica por blocos em M, P e K, mascara o acumulador a $2 \cdot DW + \lceil \log_2 K \rceil$ bits (igual ao RTL quando $K = N$) e aplica *wrap* em DW bits na saída; aceita operandos com e sem sinal; 11 testes `unittest`. `generate_rtl_vectors.py` gera os 81 casos `known`, `identity`, `zero_weights`, `numeric_limits`, `output_wrap`, `random_load`, `random_reuse`, `zero_input_reuse` e `random_reuse_again` (seed $2026 + 100 \cdot N + DW$), só $N \times N$ e com sinal.

Tabela 3: Estado de cada bloco na data deste plano.

Bloco	RTL	Verificação existente
Acelerador (`accel_systolic`, `accel_top`)	completo para $N \in \{2,4,8\}$ e $DW \in \{8,16,32\}$; sem IRQ; porta de interface e FSM + datapath no mesmo módulo (P13)	`accel_top_tb.sv`: SV dirigido, 9 instâncias paralelas, BFM AXI4-Lite com ordem AW/W e backpressure fixa, casos SLVERR/ERROR, 81 vetores (seed 2026), reset no meio de transação; roda em Vivado xsim (`run_accel.sh`), não em `xrun`; sem SVA, cobertura ou UVM
CSR do acelerador	não iniciado (contrato em `csr_readme.md`)	—
DMA	não iniciado (RTL em 17–18/11); registradores a definir	—
Memória compartilhada	não iniciada	—
Barramento / decodificador	não iniciado; só `axil_if.sv`	—
CPU RV32I	core não escolhido; entrega em 24/11	— (programas autoverificáveis planejados)
Firmware	não iniciado; entrega em 25/11	—
`soc_top`	não iniciado	—
Golden model (Python)	completo (tiling, wrap, 9 configurações)	11 testes `unittest` (`make test`); gerador de 81 vetores
Infraestrutura	—	`env_smoke` (UVM) prova Xcelium + UVM + licença; `uvm_verdict.sh`, `elab_check.sh`; `make check/test/lint/elab-smoke/sim-smoke` com `SEED` e `TEST`

1.3 Objetivos e metas da verificação

Objetivo geral. Mostrar, com evidência reproduzível (comando, seed e log), que cada bloco implementa o seu contrato (READMEs e ICD) e que o sistema integrado, a partir do firmware executado na CPU, calcula $C = A \times B$ igual ao golden model para várias dimensões e valores, com o DMA operando de forma autônoma, nas três configurações de malha, por meio de um ambiente UVM 1.2 com checagem automática e cobertura funcional, conforme o enunciado do trabalho final de verificação.

Objetivos específicos.

O1. Verificar cada unidade isoladamente antes da integração: acelerador (testbench existente, portado para `xrun` e estendido), CSR, memória, DMA e core, com testbenches do dono de cada bloco.

O2. Acelerador: equivalência bit a bit de DATA_OUT com o golden para as 9 configurações $N \times DW$, incluindo limites numéricos, *wrap*, reuso de pesos, backpressure AXI4-Lite e todos os casos negativos (SLVERR, ERROR).

O3. CSR e barramento: decodificação do mapa, semântica W1P/RW/RO, respostas de erro e ausência de efeitos colaterais.

O4. DMA: FSM completa, endereçamento de A, B e C, *tiling* (incluindo $K > N$), sequência correta de comandos ao acelerador, autonomia e tratamento de erro.

O5. Subsistema E3 (DMA + memória + CSR + acelerador, sem CPU): multiplicação ponta a ponta com o testbench no papel da CPU; regressão por seeds $\times$ N.

O6. Sistema E4: o firmware configura e dispara; C na memória igual ao golden; término por STATUS (e IRQ, se adotada); nenhuma transferência elemento a elemento pela CPU; regressão nas três configurações selecionáveis na linha de comando.

O7. Ambiente UVM conforme o enunciado: transação, biblioteca de sequences, agente com objeto de configuração, TLM, scoreboard elemento a elemento, cobertura amostrada do monitor com `cross`, testes por `+UVM_TESTNAME`, factory, veredito `TEST PASS`/`TEST FAIL`; driver A (BFM) e driver B (firmware) intercambiáveis por *factory override*; itens opcionais (reference model por TLM, monitor de barramento, latência).

O8. Medir e fechar cobertura funcional e de código por bloco, com merge da regressão no IMC e exclusões justificadas.

O9. Provar que o ambiente detecta erros (injeção de falhas com `TEST FAIL`) e que toda espera tem timeout.

O10. Medir ciclos por operação (LOAD, PROCESSING, operação completa) por configuração, para o comparativo de desempenho do relatório, sem critério de aprovação.

Fora do escopo. Temporização, área e $f_{\max}$ (cobertas pela síntese no Genus); potência; verificação física; gate-level e LEC (o Conformal está instalado, mas não é exigido); verificação formal além de lint e elaboração; conformidade completa do core RV32I (riscv-tests, riscv-arch-test, co-simulação com o Spike): o core é verificado pelo seu dono e, se for IP de terceiros, tratado como confiável, só a integração é verificada; $DW = 16$ e 32 nos níveis N2 e N3 (só no N1); saturação (o projeto define *wrap*); múltiplos domínios de clock (clock único); desempenho como critério de aprovação.

1.4 Ambiente e ferramentas

- **Simulação:** Cadence Xcelium 25.03 (`xrun -uvm`, UVM 1.2 da distribuição; o README cita 24.09 e será corrigido), SystemVerilog e SVA; formas de onda no SimVision. Flags atuais do Makefile: `-uvm -access +rwc -timescale 1ns/1ps`, `-svseed $(SEED)`, `+UVM_TESTNAME=$(TEST)`.
- **Cobertura:** Cadence IMC (Verisium Manager 25.09): `-coverage all`, merge da regressão, exclusões versionadas no repositório.
- **Modelo de referência:** golden model Python 3.9 (só biblioteca padrão) e `generate_rtl_vectors.py` (arquivos texto em hexadecimal lidos com `$fscanf`); função SystemVerilog de referência (acumulação e *wrap* em DW bits) para o aleatório restrito, validada contra os vetores Python; opcionalmente, porte em C do golden ligado por DPI-C.

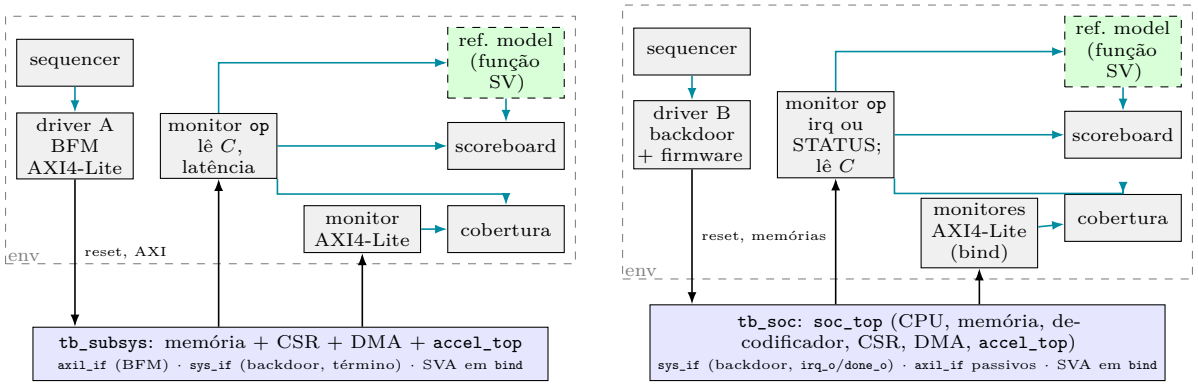
Tabela 4: Metas quantitativas (critérios de encerramento).

Métrica	Meta
Features da Tabela 6 de prioridade A com teste associado e aprovado	100 %
Features de prioridade M com teste aprovado	$\geq$ 90 % (restante justificado)
`accel_top_tb`: configurações $N \times DW$ aprovadas	9/9 (`RESUMO: 9 OK, 0 ERRO`) com seed 2026 e $\geq$ 2 seeds adicionais
Divergências acelerador × golden na regressão aleatória UVM ($\geq$ 300 operações por N com $DW = 8$; $\geq$ 50 por configuração com $DW = 16$ e 32)	0
Divergências subsistema (DMA + memória + acelerador) × golden ($\geq$ 100 operações aleatórias por N, com múltiplos tiles e $K > N$)	0
Divergências sistema (firmware) × golden ($\geq$ 50 operações aleatórias por N mais todos os dirigidos)	0
Regressão final: testes × $\geq$ 10 seeds × 3 valores de N	100 % `TEST PASS`; `UVM_ERROR : 0`; `UVM_FATAL : 0`
Operações pendentes no scoreboard ao fim de cada teste	0
Cobertura funcional (covergroups da Seção 2.3)	100 % (exclusões justificadas)
Cobertura de linha/bloco: acelerador, CSR, DMA, memória, decodificador, `soc_top`	$\geq$ 95 %
Cobertura de linha do core RV32I	$\geq$ 80 % se core próprio; excluída se IP de terceiros
Cobertura de FSM (`accel_systolic`, DMA, CSR): estados e transições	100 %
Cobertura de toggle nas portas e registradores de controle	$\geq$ 90 %
Falhas de SVA e de checkers de protocolo na regressão	0
Falhas injetadas ($\geq$ 3 mecanismos) detectadas pelo scoreboard com `TEST FAIL`	100 %
Simulações travadas (espera sem timeout) na regressão	0
Pontos de atenção P1–P19 com decisão registrada (ADR)	100 % até o fim da S2
Bugs abertos de severidade alta	0
Latência medida (ciclos) por configuração e dimensão de teste	registrada (sem critério)

- **Software:** GCC RISC-V (`-march=rv32i -mabi=ilp32`, *freestanding*); **não há toolchain RISC-V no servidor da disciplina**, por isso o `.hex` é versionado em `06.Firmware/`; header do mapa de registradores gerado do ICD.
- **Lint:** Verible (`.rules.verible_lint`: `always-ff-non-blocking`, `signal-name-style`, parâmetros em maiúsculas, linhas de 100 colunas) a cada commit; elaboração com `elab_check.sh` (falha em `*E`, `*F` e `*W,CUVMPW`).
- **Complementar:** Vivado xsim só como legado do testbench unitário até o porte; Genus 25.1 para a síntese (fora do escopo); `uvm_verdict.sh` (`SIM PASS` com `UVM_ERROR : 0` e `UVM_FATAL : 0`); novos alvos do Makefile `sim-accel`, `sim-subsys`, `sim-soc`, `regress` e `cov-merge`.

A Figura 2 mostra os dois ambientes UVM. Os componentes são escritos uma vez e reaproveitados: a transação, o monitor, o modelo de referência, o scoreboard e a cobertura são os mesmos no subsistema e no sistema; só o driver muda, por *factory override*. A Tabela 5 relaciona cada requisito do enunciado ao componente que o atende.

Infraestrutura que ainda não existe e é pré-requisito. (i) Alvo `make sim-accel` com o testbench do acelerador em `xrun` e cobertura (hoje só Vivado). (ii) Agente AXI4-Lite UVM reaproveitável (driver, monitor, config e SVA em `bind`) em `05.Verification/common/vip/`. (iii) Gerador de casos (M, K, P) com imagem de memória e bloco de parâmetros, além dos 81 vetores $N \times N$. (iv) Função SystemVerilog de referência e, opcionalmente, porte em C do golden para DPI-C. (v) RTL do CSR e modelos comportamentais de DMA e CPU (sprint 2.3 do roadmap), para começar antes do RTL real. (vi) `top_tb` com interface de *backdoor* para as memórias e sinal observável de término (P8). (vii) Script de regressão (testes × seeds × N) com merge no IMC e tabela de vereditos. (viii) Firmware genérico, header gerado do ICD e `.hex` versionado.



(a) Subsistema E3: o testbench faz o papel da CPU. **(b)** Sistema E4: driver B por *factory override*.

Figura 2: Ambientes UVM. Cinza: componentes UVM; verde tracejado: modelo de referência (opcional como componente, obrigatório como função do scoreboard); azul: módulos do `top_tb`. Setas em ciano são conexões TLM. As únicas referências hierárquicas ficam no `top_tb` e nas interfaces; as classes só enxergam o *virtual interface*.

Tabela 5: Requisitos do enunciado do trabalho final e componentes que os atendem.

Requisito	Componente previsto	Nível
Transação	`op_item` (`uvm_sequence_item`): `rand` M, K, P, A, B, bases e seed, com constraints; `convert2string` e `do_compare`; campos de resposta C observada e `latency_cycles`.	N2, N3
Sequences	`op_seq_lib`: `op_smoke_seq`, `op_vectors_seq` (lê os arquivos do golden), `op_tiling_seq`, `op_rand_seq`, `op_errors_seq`, com `start_item`/`randomize`/`finish_item`.	N2, N3
Sequencer	`op_sequencer` (`uvm_sequencer#(op_item)`), criado pelo agente e conectado ao driver no `connect_phase`.	N2, N3
Driver	`op_driver_bfm` (A: programa DMA e CSR por AXI4-Lite) e `op_driver_fw` (B: carrega memórias por backdoor com o sistema em reset, libera o reset, espera o término); vif obtido no `uvm_config_db`.	N2 / N3
Monitor	`op_monitor`: detecta início e fim (START no barramento ou marcador; `irq` ou STATUS.DONE), lê C da memória por backdoor, mede a latência e publica no `analysis_port`; não consulta driver nem sequence.	N2, N3
Agente	`op_agent` com `op_cfg` (vif, `is_active`, `ARRAY_SIZE`, `DATA_WIDTH`, timeout).	N2, N3
TLM	`seq_item_port`; `analysis_port` do monitor para `op_ref_model`, `op_scoreboard` (`uvm_tlm_analysis_fifo`) e `op_coverage`; nenhum acesso por handle direto.	N2, N3
Scoreboard	`op_scoreboard`: compara C elemento a elemento, `uvm_error` com linha, coluna, esperado e obtido; `check_phase` acusa pendências; resumo final.	N2, N3
Cobertura funcional	`op_coverage` e `axil_coverage` (`uvm_subscriber`), amostradas das transações do monitor; Tabela 7.	N2, N3
Env e testes	`subsys_env`/`soc_env`; `op_base_test` (`+UVM_TESTNAME`, `+ARRAY_SIZE`, `+ERR_INJECT`, timeout) e testes T09–T24; criação pela factory.	N2, N3
Execução	Makefile: `sim-subsys`, `sim-soc`, `regress` com `SEED`, `TEST`, `N` e `COV=1`; veredito `TEST PASS`/`TEST FAIL` no `report_phase`, validado por `uvm_verdict.sh`.	todos
Opcionais	`op_ref_model` por TLM; `axil_agent` passivo (monitor de barramento e autonomia); `+ARRAY_SIZE` com três valores na regressão; `latency_cycles` na transação.	N2, N3

2 Plano de teste

2.1 Features a serem verificadas

A Tabela 6 lista as features extraídas do RTL, dos READMEs e do enunciado. Métodos: **D** = teste dirigido, **R** = aleatório restrito, **A** = asserção, **M** = comparação com modelo de referência, **C** = cobertura funcional, **G** = netlist (LEC ou gate-level), **F** = checagem estrutural (lint, elaboração, revisão). Prioridade: **A**lta, **M**édia, **B**aixa. Os identificadores Pn remetem aos pontos de atenção da Seção 2.1.1.

Tabela 6: Features a serem verificadas.

ID	Feature	Comportamento esperado / critério	Método	Pri.
Acelerador (`accel_top` / `accel_systolic`)				
ACC-01	Reset	Com `rst_ni`=0: `status_o`=0 (IDLE, DONE=0, ERROR=0), `awready`=`wready`=`arready`=0, `bvalid`=`rvalid`=0, `input_q`, `weight_q` e `result_q` zerados. Reset assíncrono com AW aceito sem W, ou com `rvalid` pendente, descarta a transação; depois escrita e leitura funcionam; sem X nas saídas após o reset.	D, A	A
ACC-02	Handshake AXI4-Lite	Transferência quando VALID e READY estão em 1 na borda; AW e W aceitos em qualquer ordem (W antes de AW, AW antes de W, juntos) e de forma independente; a escrita efetiva uma única vez (`write_commit`) após ambos; `bvalid` sobe exatamente 1 ciclo após o último handshake; `rvalid` sobe 1 ciclo após o handshake de AR.	D, R, A	A
ACC-03	Uma pendente por canal; back-pressure	`awready`/`wready`=0 enquanto houver AW/W pendente ou `bvalid`=1; `arready`=0 enquanto `rvalid`=1; `bresp`/`bvalid` e `rdata`/`rresp`/`rvalid` estáveis até `bready`/`rready` (atraso de 0 a n ciclos).	D, R, A	A
ACC-04	Decodificação de DATA_IN	Escrita alinhada em [`DATA_IN_BASE`, $+N^2 \cdot DW/8$) com STATE=IDLE: OKAY e `input_q[(awaddr-base)»2]` atualizado; elementos em little-endian ($32/DW$ por palavra), posição (endereço − base)/($DW/8$).	D, R, M	A
ACC-05	Byte strobes	Só os bytes com `wstrb[lane]`=1 são escritos; `wstrb`=0 responde OKAY sem efeito; reconstrução byte a byte de uma palavra; strobes parciais em palavras diferentes de 0.	D, R	A
ACC-06	Erros de escrita	SLVERR (`2'b10`) para `awaddr[1:0]`≠0, offset ≥ `BUFFER_BYTES`, endereço abaixo da base, escrita em DATA_OUT ou STATE≠IDLE (LOADING/PROCESSING); buffer preservado; `status_o[3]` (ERROR) não é afetado.	D, R	A
ACC-07	Leitura de DATA_OUT	Leitura alinhada em [`DATA_OUT_BASE`, +`BUFFER_BYTES`): OKAY com `rdata`=`output_words[idx]`, capturado no handshake de AR; empacotamento igual ao de DATA_IN.	D, M	A
ACC-08	Erros de leitura; `awprot`/`arprot`	SLVERR com `rdata`=0 para desalinhado, fora da faixa ou em DATA_IN; leituras aceitas em qualquer estado; `awprot`/`arprot` sem efeito na resposta (P7).	D, R	A
ACC-09	Comando LOAD	Aceito só com IDLE, `mode_i`=0, `load_i`=1, `start_i`=0; vai a LOADING e limpa DONE no ciclo da aceitação; `weight_q[r][*]` ← `data_i[r*N+*]` no ciclo r; LOADING dura N ciclos; volta a IDLE com DONE=1.	D, M, A, C	A
ACC-10	Comando START	Aceito só com IDLE, `mode_i`=1, `start_i`=1, `load_i`=0; PROCESSING dura $3N-2$ ciclos (`LAST_CYCLE`=$3N-3$); DONE=1 e IDLE ao final; DONE=1 não bloqueia novo comando.	D, A, C	A
ACC-11	Comando inválido	`load_i`/`start_i` com `mode_i` errado, ambos em 1, ou fora de IDLE (LOAD ou START durante LOADING e durante PROCESSING): ERROR=1 pegajoso, sem mudança de estado, de DONE nem do resultado em curso.	D, R, A	A
ACC-12	CLR_ERROR	`clr_error_i`=1 limpa ERROR; se coincidir com comando inválido no mesmo ciclo, o *set* prevalece (ERROR permanece 1), a confirmar no ICD (P5).	D, A	M

continua na próxima página

Tabela 6: Features a serem verificadas (continuação).

ID	Feature	Comportamento esperado / critério	Método	Pri.
ACC-13	Produto de um tile	DATA_OUT igual ao golden para os casos `known`, `identity`, `zero_weights` e `random_load` nas 9 configurações $N \in \{2,4,8\} \times DW \in \{8,16,32\}$.	D, R, M	A
ACC-14	Sinal, limites e wrap	Complemento de dois; $(-2^{DW-1}) \times (-2^{DW-1})$, máx × máx, máx × mín; acumulador `ACC_WIDTH` $= 2 \cdot DW + \lceil \log_2 N \rceil$; saída = DW bits baixos de `mac[N-1][col]` (*wrap*, sem saturação), igual ao golden (`numeric_limits`, `output_wrap`).	D, M	A
ACC-15	Persistência e troca de pesos	Após um LOAD, STARTs consecutivos com A diferentes usam os mesmos pesos (`random_reuse`, `zero_input_reuse`, `random_reuse_again`); novo LOAD substitui todos os pesos; reset os zera.	D, R, M	A
ACC-16	Latência do resultado; leitura em PROCESSING	$C[i][j]$ é capturado em `result_q` no ciclo $i + N - 1 + j$ de PROCESSING; leitura de DATA_OUT durante PROCESSING retorna mistura de valores antigos e novos (documentar; o DMA nunca lê antes de DONE).	D, A, C	M
ACC-17	Janela escrita × comando	Escrita que efetiva no mesmo ciclo em que LOAD/START é aceito (STATE ainda IDLE) é aceita e entra na operação; um ciclo depois recebe SLVERR. Decidir se é permitido (P4) e cobrir os dois casos.	D, A, C	M
ACC-18	Parametrização	Elaboração falha para $N \notin \{2,4,8\}$, $DW \notin \{8,16,32\}$ e AXI $\neq 32/32$ (módulos `ACCEL_*_MUST_BE_*`); `BUFFER_WORDS`, `WORD_INDEX_WIDTH` e faixas coerentes em cada configuração; lint limpo.	D, F	M
ACC-19	Sem transferência pelo host durante a operação	Durante LOADING/PROCESSING o único tráfego legal é leitura (STATUS via CSR e DATA_OUT); escritas recebem SLVERR.	A, C	M
ACC-20	Ciclos por operação	LOAD = N, PROCESSING = $3N - 2$, medidos pelo monitor em cada N e registrados para o relatório.	D	B
CSR do acelerador (contrato `csr_readme.md`)				
CSR-01	Mapa e reset	CTRL em `0x1000_0000` (RW) e STATUS em `0x1000_0004` (RO); outros offsets: resposta de erro (a definir); reset: MODE=0, sem pulsos.	D, R	A
CSR-02	CTRL.MODE e reservados	Bit 0 RW mantém o valor e alimenta `mode_i`; leitura devolve o valor; bits 31:4 ignorados na escrita e lidos como 0.	D	A
CSR-03	LOAD, START, CLR_ERROR (W1P)	Escrita de 1 nos bits 1–3 gera pulso de exatamente 1 ciclo em `load_i`/`start_i`/`clr_error_i`, uma vez por transação; escrita de 0 não gera pulso; leitura devolve 0; MODE escrito na mesma transação vale no ciclo do pulso.	D, A, C	A
CSR-04	STATUS	Reflete `status_o[31:0]` (atraso de 0 ou 1 ciclo, a definir); escrita em STATUS sem efeito (resposta a definir).	D, A	A
CSR-05	Escritas combinadas	Mesma transação com LOAD+START, ou CLR_ERROR junto com comando inválido: comportamento do acelerador (ERROR) documentado e coberto (P5).	D, C	M
CSR-06	Interrupção (condicional, 29/11)	Se adotada: `irq` em DONE/ERROR, habilitação e limpeza (W1C) conforme o ICD; nenhum pulso espúrio no reset.	D, A	M
DMA (seção 5 do enunciado; registradores a definir no ICD)				
DMA-01	Registradores	Bases de A, B e C, dimensões (M, K, P ou equivalentes), CTRL (START) e STATUS (BUSY, DONE, ERR); leitura devolve o escrito; escrita com BUSY ignorada ou rejeitada (a definir); offsets inválidos respondem erro.	D, R	A
DMA-02	FSM	Estados de leitura de A, leitura de B, comando e espera do acelerador, leitura de DATA_OUT e escrita de C, com contadores de endereço e de tiles; todos os estados e transições exercitados; retorno a IDLE ao final e após erro.	D, A, C	A

continua na próxima página

Tabela 6: Features a serem verificadas (continuação).

ID	Feature	Comportamento esperado / critério	Método	Pri.
DMA-03	Leitura de A e B	Sequência de endereços = base + layout (row-major, a definir) × dimensões; $N^2 \cdot DW/32$ palavras por tile; dados entregues a DATA_IN na ordem do acelerador (pesos = tile de B, ativações = tile de A), palavras de 32 bits com `wstrb=4'hF`.	D, R, M	A
DMA-04	Sequência de comandos ao acelerador	Por tile: DATA_IN ← B, MODE=0, LOAD, esperar DONE; DATA_IN ← A, MODE=1, START, esperar DONE; ler DATA_OUT. Nunca LOAD/START fora de IDLE, nunca escrita em DATA_IN fora de IDLE, nunca leitura de DATA_OUT antes de DONE. Via CSR ou sinais diretos (P9).	D, A	A
DMA-05	Escrita de C	Tile gravado em base de C + deslocamento correto; nenhuma escrita fora da região $[\text{base}_C,\ \text{base}_C + M \cdot P \cdot DW/8)$.	D, M, A	A
DMA-06	Tiling	$M, P > N$ (vários tiles de saída) e $K > N$ (soma de parciais módulo 2^{DW}, P1): C igual ao golden; cobertura de tiles em M, P, $K \in \{1, 2, >2\}$; dimensões não múltiplas de N conforme a decisão P12 (padding com zero ou ERR).	D, R, M, C	A
DMA-07	Autonomia	Após CTRL.START nenhuma transação do host é necessária até DONE; todo o tráfego para memória e acelerador na janela é do DMA.	A, D	A
DMA-08	Término	STATUS.DONE (e `irq`, se houver) só após a última escrita de C aceita pela memória; BUSY=1 durante a operação; DONE limpo em novo START.	D, A	A
DMA-09	Erros	SLVERR/RRESP do acelerador, dimensões zero ou acima do máximo, START com BUSY: STATUS.ERR, parada limpa, sem escrita parcial fora de C; recuperação (a definir).	D	M
DMA-10	Concorrência com a CPU	CPU lendo STATUS ou a memória de dados durante a operação não corrompe nem trava o DMA (arbitragem/porta a definir).	R, A	M
DMA-11	Ciclos por operação	Ciclos entre START e DONE por N e por dimensão; registrados.	D	B
Memória compartilhada				
MEM-01	Acesso por palavra com byte enables	Escrita e leitura de 32 bits com byte enables em cada porta; latência definida no ICD; o dado lido é o último escrito.	D, R, A	A
MEM-02	Duas portas	Acessos simultâneos em endereços distintos corretos; mesmo endereço no mesmo ciclo com comportamento definido no ICD (read-old/read-new).	D, R	A
MEM-03	Carga e leitura por backdoor	O testbench carrega firmware e dados e lê C por tarefas no `top_tb`/interface; sem `initial`/`$readmemh` no RTL; acesso fora da faixa com comportamento definido.	D	A
CPU e firmware				
CPU-01	Execução do firmware	Core RV32I busca a partir do vetor de reset (a definir) e executa `lw`/`sw` em MMIO; programas autoverificáveis do dono (aritmética, loads/stores, desvios, MMIO).	D	A
CPU-02	Firmware genérico	Lê o bloco de parâmetros em endereço fixo (bases de A, B, C; M, K, P); programa DMA e CSR por ponteiros `volatile` do header; dispara START; um único `.hex`.	D, M	A
CPU-03	Término	Polling de STATUS (ou tratador de IRQ) e escrita de um marcador de fim (palavra no bloco de parâmetros, a definir) visível ao monitor.	D	A
CPU-04	Sem transferência por software	Entre START e DONE a CPU não escreve na região de C nem acessa DATA_IN/DATA_OUT; só lê STATUS.	A, D	A
CPU-05	Várias operações por simulação	Firmware em laço: espera novo bloco de parâmetros, executa, sinaliza (a definir).	D	M
Sistema (`soc_top`)				
SYS-01	Mapa de endereços	Cada região da Tabela 2 alcançada pelo iniciador correto; fora do mapa, resposta definida; sem colisões.	D, A	A

continua na próxima página

Tabela 6: Features a serem verificadas (continuação).

ID	Feature	Comportamento esperado / critério	Método	Pri.
SYS-02	Multiplicação ponta a ponta	Com firmware: C na memória igual ao golden para $N \times N$, múltiplos tiles e $K > N$, com dados conhecidos, identidade, zeros, limites e aleatórios ($DW = 8$).	D, R, M	A
SYS-03	Sinalização de término	Firmware vê STATUS.DONE; se IRQ adotada, chega à CPU e o tratador conclui; sem término falso antes da última escrita de C.	D, A	A
SYS-04	Autonomia no sistema	Monitor de barramento: entre START e DONE só leituras de STATUS pela CPU; todo o resto é do DMA.	A, C	A
SYS-05	Três configurações	Regressão em $N \in \{2, 4, 8\}$ ($DW = 8$), N selecionado na linha de comando (`-defparam`/`+define`), mesmo firmware.	D, R, C	A
SYS-06	Operações consecutivas	Duas ou mais operações seguidas sem reset, com dimensões e pesos diferentes; sem estado residual (pesos antigos, DONE antigo).	D, R, M	M
SYS-07	Reset no meio da operação	Sistema volta ao estado inicial; nova operação correta.	D	M
SYS-08	Latência total	Ciclos entre START e DONE por configuração e dimensão; registrados.	D	B
SYS-09	Sintetizabilidade	Lint Verible limpo e elaboração do `soc_top` nas 3 configurações; sem `initial`/`$readmemh` no RTL.	F	M
Ambiente UVM (requisitos do trabalho final)				
UVM-01	Transação e sequences	`op_item` (`uvm_sequence_item`) com `rand` M, K, P, A, B, endereços e constraints; `convert2string` e `do_compare`; biblioteca de sequences dirigidas e aleatórias com `start_item/randomize/finish_item`.	F	A
UVM-02	Agente e TLM	`uvm_agent` com sequencer, driver (vif via `uvm_config_db`), monitor passivo e objeto de configuração (vif, `is_active`, `ARRAY_SIZE`, timeout); toda comunicação por TLM (`seq_item_port`, `analysis_port`, `uvm_tlm_analysis_fifo`).	F	A
UVM-03	Scoreboard	Compara C elemento a elemento; `uvm_error` com linha, coluna, esperado e obtido; respeita o *wrap* de DW bits; `check_phase` acusa operações pendentes; resumo final (comparadas, acertos, divergências).	D	A
UVM-04	Cobertura e veredito	Covergroups amostrados do monitor (nunca do driver), com pelo menos um `cross`, habilitáveis na linha de comando e visíveis no IMC; testes por `+UVM_TESTNAME` e `-svseed`; `TEST PASS`/`TEST FAIL` explícitos no log.	D, C	A
UVM-05	Dois drivers por factory override	Driver A (BFM AXI4-Lite que programa DMA e CSR) e driver B (backdoor nas memórias + reset + firmware) com a mesma transação, o mesmo monitor e o mesmo scoreboard.	D	A
UVM-06	Detecção de erro e timeout	Injeção de falha (`` `define `` ou plusarg) faz o scoreboard acusar e o teste terminar em `TEST FAIL`; toda espera tem timeout e gera `uvm_error`/`uvm_fatal`, nunca simulação travada.	D	A
UVM-07	Reference model por TLM (opcional)	Componente separado recebe a transação do monitor e publica C esperada ao scoreboard (função SV ou DPI-C).	M	M
UVM-08	Monitor de barramento e latência (opcional)	Agente passivo reconstrói transações AXI4-Lite internas (autonomia) e o monitor registra `latency_cycles` na transação.	A, C	M

2.1.1 Pontos de atenção levantados na revisão do RTL

A leitura de `accel_top.sv`, `accel_systolic.sv`, `accel_top_tb.sv`, do golden model e dos READMEs apontou comportamentos que o plano deve confirmar ou que a equipe precisa decidir. Cada um vira teste dirigido, asserção, item de cobertura ou ADR.

P1. Acumulação de tiles em K fora do acelerador. A saída é truncada a DW bits (`result_q` recebe os DW bits baixos de `mac[N-1][col]`) e não há entrada de soma parcial nem modo de

acumulação. Para $K > N$ as parciais têm de ser somadas fora (DMA com leitura-modificação-escrita de C, módulo 2^{DW}) ou o acelerador ganha um modo ACCUMULATE. É a decisão que define a FSM do DMA e o ICD. O golden já acumula por bloco com máscara e *wrap*, então qualquer das opções fecha com ele desde que a soma seja módulo 2^{DW} (ACC-14, DMA-06).

P2. Largura do acumulador e de C. `ACC_WIDTH` $= 2 \cdot DW + \lceil \log_2 N \rceil$ internamente, mas só os DW bits baixos saem; o enunciado pede largura "explicitamente definida e justificada". Decidir se C fica em DW bits com *wrap* (como hoje e como o golden) ou se DATA_OUT expõe o acumulador completo (muda o tamanho do buffer de saída, o DMA e o golden).

P3. Leituras de DATA_OUT durante PROCESSING. `output_words` é apelido de `result_q`, atualizado ao longo de PROCESSING (ciclo $i + N - 1 + j$), e `read_allowed` não depende do estado. O README admite "resultados incompletos". Asserção no DMA: nenhuma leitura de DATA_OUT antes de DONE; cobertura do caso (ACC-16, DMA-04).

P4. Janela de corrida entre escrita e comando. `write_allowed` olha `status_o[1:0]==IDLE` no ciclo do `write_commit`: uma escrita que efetiva no mesmo ciclo da aceitação de LOAD/START é aceita e entra na operação; um ciclo depois recebe SLVERR. O testbench atual só cobre o segundo caso. Definir no ICD se o DMA garante o último BRESP antes do comando e cobrir com asserção (ACC-17).

P5. CLR_ERROR e comando inválido no mesmo ciclo. O *set* prevalece (`if (clr_error_i) error_q <= 0; if (invalid_command) error_q <= 1;`). Com W1P isso acontece se o firmware escrever CLR_ERROR+START com MODE errado numa só transação. Documentar e testar (ACC-12, CSR-05).

P6. Erros AXI não ligam ERROR. SLVERR vai só em BRESP/RRESP; `status_o[3]` é só para comando inválido. O DMA precisa checar BRESP/RRESP e expor STATUS.ERR, senão o firmware nunca saberá (ACC-06, DMA-09).

P7. `awprot/arprot` ignorados. Permitido em AXI4-Lite; registrar como decisão e cobrir que não alteram a resposta (ACC-08).

P8. Sem interrupção e sem sinal de término observável. `accel_top` não tem `irq`; a decisão é de 29/11. O monitor UVM precisa detectar o fim sem referência hierárquica fora do `top_tb`: pino `irq_o`/`done_o` no `soc_top`, ou leitura de STATUS observada por interface ligada no `top_tb` (CSR-06, SYS-03).

P9. Quem emite LOAD/START por tile e como o DMA vê DONE. O enunciado pede "sinais de requisição, pronto/ocupado e conclusão" e autonomia após o início; hoje só existem `status_o` e o slave AXI4-Lite, e o CSR é descrito como do host. Com *tiling*, a CPU não pode ficar no laço: o DMA deve emitir MODE/LOAD/START (escrevendo o CSR pelo barramento ou por fios diretos) e observar `status_o`. Definir no ICD (DMA-04, DMA-07).

P10. CSR não implementado e mapa de memória incompleto e inconsistente. `csr_readme.md` dá a faixa de CSR em `0x0000_0000`–`0x4000_0000` e o acelerador em `0x1000_0000`–`0x2000_0000`; os buffers em `0x4000_1000`/`0x4000_2000` ficam fora; DMA "to be defined"; memórias e vetor de reset sem base; dono do CSR indefinido. Tudo precisa fechar na S1: o firmware (header), o decodificador e o ambiente dependem disso (CSR-01, SYS-01).

P11. Buffer único DATA_IN para pesos e ativações. A sequência é obrigatoriamente $B \rightarrow$ LOAD $\rightarrow A \rightarrow$ START; escritas em LOADING retornam SLVERR, então não há sobreposição (*double-buffering*). Impacto de desempenho a registrar no comparativo (DMA-04, DMA-11).

P12. Dimensões não múltiplas de N. O acelerador só faz $N \times N$; o golden aceita qualquer M, K, P. Decidir: o DMA faz *padding* com zeros, o firmware exige múltiplos, ou STATUS.ERR (DMA-06).

P13. Violações das convenções do time. `accel_top` usa porta de interface (`axil_if.target s_axi`) e a regra é porta plana; `accel_systolic` junta FSM e datapath (regra `_ctrl`/`_dp`). Waiver por ADR ou refatorar antes do E3, com o testbench como rede de proteção (ACC-18).

P14. Toolchain do testbench unitário. `run_accel.sh` usa Vivado (`xvlog`/`xelab`/`xsim`) e `rg`; o projeto usa `xrun` (Xcelium 25.03 instalado; o README diz 24.09). Portar para `xrun` e criar `make sim-accel` é pré-requisito de regressão e cobertura.

P15. Seeds e veredito do testbench atual. `SEED ?= 1` no Makefile, 2026 fixo no gerador e no `$display`; o testbench aborta no primeiro erro (`$fatal`) e a linha `RESUMO: 9 OK, 0 ERRO` é fixa, não contada. Parametrizar a seed e contar erros conforme `CONTRIBUTING.md`.

P16. Lacunas do testbench atual. Não cobre LOAD durante PROCESSING (só START), leitura em PROCESSING, backpressure aleatório (fixo em 3 ciclos), CLR_ERROR + inválido, strobes parciais em palavra diferente de 0, `awprot`; sem SVA nem cobertura (ACC-03, ACC-11, ACC-16).

P17. Persistência dos pesos entre operações do sistema. `weight_q` só muda em LOADING ou no reset: operação consecutiva com B diferente exige novo LOAD por tile; o DMA deve sempre recarregar (ou o reuso é coberto explicitamente) (ACC-15, SYS-06).

P18. Vetores do golden só $N \times N$. `generate_rtl_vectors.py` gera 81 casos quadrados de um tile; para N2/N3 é preciso um gerador (M, K, P) com imagem de memória e bloco de parâmetros, e um modelo "online" (função SV ou DPI-C) para o aleatório restrito.

P19. Larguras internas diferentes, resultado igual. Golden: $2 \cdot DW + \lceil \log_2 K \rceil$ bits, calculados com `bit_length()` de $K - 1$; RTL: $2 \cdot DW + \lceil \log_2 N \rceil$. Coincidem para $K = N$ e, após o *wrap*, dão o mesmo C em qualquer caso, mas só enquanto C ficar em DW bits (P2).

2.2 Estratégias de inserção de estímulos

O plano combina sete estratégias, da mais controlada à mais abrangente.

E1. Testes dirigidos. Tasks SystemVerilog (N1) e sequences UVM dirigidas (N2/N3): reset em cada estado, cada resposta AXI4-Lite, cada comando válido e inválido, reuso de pesos, backpressure em cada canal; no sistema, uma operação $N \times N$ conhecida.

E2. Vetores do golden model em arquivo. Os 81 casos de `generate_rtl_vectors.py` (seed 2026 e seeds extras) no N1; extensão do gerador com casos (M, K, P), imagem de memória e bloco de parâmetros para N2/N3. A sequence lê A e B; o scoreboard lê a C esperada.

E3. Aleatório restrito UVM. `op_item` com `rand` $M, K, P \in \{N, 2N, 3N\}$ (e $N + 1$ se P12 permitir), dados com peso nos extremos (-2^{DW-1}, -1, 0, $+1$, $2^{DW-1} - 1$), bases alinhadas e não sobrepostas, número de operações por teste, atrasos e ordem AW/W e backpressure do BFM; seeds explícitas (`-svseed`), uma lista de seeds por teste na regressão.

E4. Firmware genérico com bloco de parâmetros. Um único `.hex` versionado; o driver B escreve A, B e o bloco de parâmetros por *backdoor* com o sistema em reset, libera o reset e espera o marcador de fim (alternativa A do enunciado: carga das memórias por *backdoor*). Firmware por teste só no smoke de boot.

E5. Casos negativos e injeção de erros. SLVERR, comandos inválidos, dimensões inválidas, START com BUSY, reset no meio; injeção deliberada (`+ERR_INJECT=<tipo>` ou `` `define ``) que corrompe um elemento de C, uma dimensão, ou faz a CPU escrever em C, para provar que o scoreboard e o monitor de autonomia acusam e o teste termina em `TEST FAIL`.

E6. Backpressure e concorrência no barramento. `bready`/`rready` atrasados de 0 a 7 ciclos, AW/W em todas as ordens, polling de STATUS pela CPU ou pelo BFM durante a operação do DMA.

E7. Múltiplas configurações. $N \in \{2, 4, 8\}$ ($DW = 8$) nos níveis N2/N3 e as 9 configurações no N1, por parâmetro de elaboração na linha de comando; a regressão repete cada teste por N.

Verificação da resposta (checking).

- **Scoreboard:** compara C elemento a elemento (linha, coluna, esperado, obtido) com *wrap* em DW bits; conta operações, acertos e divergências; o `check_phase` acusa itens sem par; resumo com `UVM_NONE`. O valor esperado nunca vem da saída do DUT.
- **Modelo de referência:** função SystemVerilog de referência (acumulação e *wrap*) no componente `op_ref_model`, ligado por TLM (opcional: porte em C do golden via DPI-C); arquivos do golden Python para os dirigidos e para validar a função SV nos mesmos casos.
- **SVA em `bind`:** protocolo AXI4-Lite (VALID estável até READY, sinais estáveis, sem X, uma transação pendente, resposta só após o handshake); FSM do acelerador (transições legais, N e $3N - 2$ ciclos, DONE/ERROR); W1P do CSR (pulso de 1 ciclo); DMA (sem LOAD/START fora de IDLE,

sem leitura de DATA_OUT antes de DONE, sem escrita fora de C); autonomia (nenhuma escrita da CPU em C ou DATA_IN entre START e DONE).

- **Monitor de barramento passivo:** reconstrói transações AXI4-Lite internas para autonomia, cobertura e latência.
- **Checagens de fim de teste:** scoreboard vazio; agentes sem transações pendentes (AW sem W, B/R não aceitos); STATUS final IDLE/DONE com ERROR=0 (salvo nos negativos); `UVM_ERROR : 0` e `UVM_FATAL : 0` implicam `TEST PASS`, impresso no `report_phase` e validado por `uvm_verdict.sh`.

2.3 Modelo de cobertura

Tabela 7: Grupos de cobertura funcional.

Covergroup	Coverpoints / bins principais
`cg_cfg`	N {2, 4, 8} × DW {8, 16, 32} (N1); N com $DW = 8$ (N2/N3); **cross** N× número de tiles.
`cg_axil` (por agente)	canal (AW/W/AR) × resposta (OKAY, SLVERR); ordem AW/W (AW primeiro, W primeiro, juntos); atraso de `bready`/`rready` {0, 1, 2–4, >4}; alinhamento; região (DATA_IN, DATA_OUT, CSR, DMA, fora); `wstrb` {`0xF`, um byte, parcial, 0}; **cross** região × resposta; **cross** estado do acelerador × tipo de acesso (escrita em IDLE/LOADING/PROCESSING).
`cg_accel_cmd`	comando {LOAD, START, ambos, nenhum} × `mode_i` × estado {IDLE, LOADING, PROCESSING} → aceito/inválido; CLR_ERROR {com ERROR=1, com ERROR=0, simultâneo a inválido}; sequência {LOAD→START, START→START (reuso), LOAD→LOAD}; transições da FSM; **cross** comando × estado.
`cg_accel_data`	A e B por elemento {mín, −1, 0, +1, máx, outros}; *wrap* ocorrido (sim/não); leitura de DATA_OUT durante PROCESSING; **cross** mín×mín, máx×máx, mín×máx.
`cg_csr`	campo escrito {MODE, LOAD, START, CLR_ERROR, combinações}; leitura de STATUS em cada estado; acesso a offset reservado; **cross** campo × estado.
`cg_dma_op` (do monitor)	M/N, P/N, $K/N \in \{1, 2, >2\}$; M mod N, K mod N, P mod $N \in \{0, \neq 0\}$; bases {baixa, alta, adjacentes}; operações consecutivas {1, 2–4, >4}; erro do DMA {nenhum, dimensão, SLVERR}; **cross** $\text{tiles}_M \times \text{tiles}_P \times \text{tiles}_K$.
`cg_sys`	término {polling, irq}; latência (bins por faixa, por N); CPU acessando a memória de dados durante a operação (sim/não); injeção de erro ativa; **cross** N× dimensões × término.

Os covergroups são amostrados em `uvm_subscriber`s ligados ao `analysis_port` dos monitores, nunca a partir do item do driver. A cobertura de código (linha, desvio, expressão, FSM e toggle) é medida por bloco e consolidada no IMC; exclusões (código só de simulação, IP de terceiros) ficam registradas com justificativa no repositório.

2.4 Resumo dos testes planejados

Tabela 8: Lista de testes e rastreabilidade com as features.

Teste	Nome	Estímulo	Features cobertas
Unidade			
T01	`golden_unittest` (`make test`)	E2	base de ACC-13, ACC-14 e DMA-06
T02	`accel_unit_tb` (portado para `xrun`, 9 configurações)	E1, E2	ACC-01 a ACC-07, ACC-09 a ACC-11, ACC-13 a ACC-15, ACC-18
T03	`accel_unit_gaps` (leitura em PROCESSING, LOAD em PROCESSING, CLR_ERROR + inválido, janela de corrida, backpressure aleatório, strobes parciais; 3 seeds)	E1, E3, E5, E6	ACC-03, ACC-05, ACC-08, ACC-11, ACC-12, ACC-16, ACC-17
T04	`csr_unit`	E1, E5	CSR-01 a CSR-05
T05	`mem_unit`	E1, E3	MEM-01 a MEM-03

continua na próxima página

Tabela 8: Lista de testes e rastreabilidade com as features (continuação).

Teste	Nome	Estímulo	Features cobertas
T06	`dma_unit` (memória e acelerador comportamentais)	E1, E3	DMA-01 a DMA-03, DMA-05, DMA-09
T07	`cpu_programs`	E1	CPU-01
Acelerador (bloco, UVM)			
T08	`accel_uvm_rand` (agente AXI4-Lite + comandos, 9 configurações, cobertura)	E3, E6, E7	ACC-02 a ACC-08, ACC-09 a ACC-17, ACC-19, ACC-20
Subsistema (E3, driver A)			
T09	`subsys_smoke` (uma operação $N \times N$ conhecida)	E1	DMA-01 a DMA-05, DMA-07, DMA-08, UVM-01 a UVM-04
T10	`subsys_vectors` (casos do golden por N)	E2	DMA-03 a DMA-06, ACC-13 a ACC-15
T11	`subsys_tiling` ($M, P > N$; $K > N$; não múltiplos conforme P12)	E1, E2	DMA-05, DMA-06
T12	`subsys_rand` (várias operações, backpressure, 3 valores de N)	E3, E6, E7	DMA-02 a DMA-08, DMA-10, `cg_dma_op`
T13	`subsys_errors` (dimensões inválidas, SLVERR, START com BUSY, reset no meio)	E5	DMA-09, SYS-07
T14	`subsys_autonomy_latency`	E1, E3	DMA-07, DMA-08, DMA-11, UVM-08
Sistema (E4, driver B)			
T15	`soc_boot_smoke` (`.hex` fixo, sem DMA)	E4	CPU-01, CPU-03, SYS-01, MEM-03
T16	`soc_matmul_directed` (firmware genérico, $N \times N$ conhecida)	E4, E1	SYS-02, SYS-03, CPU-02, UVM-05
T17	`soc_matmul_vectors`	E4, E2	SYS-02, CSR-01 a CSR-04
T18	`soc_matmul_rand` (múltiplas operações, 3 valores de N)	E4, E3, E7	SYS-02, SYS-05, SYS-06, CPU-05, `cg_sys`
T19	`soc_autonomy` (monitor de barramento)	E4	SYS-04, CPU-04, ACC-19
T20	`soc_irq` (condicional à decisão de 29/11)	E4	SYS-03, CSR-06
T21	`soc_reset_midop`	E4, E5	SYS-07
T22	`soc_error_injection` (`+ERR_INJECT` → `TEST FAIL`)	E5	UVM-03, UVM-06
T23	`soc_timeout` (operação que não termina)	E5	UVM-06
T24	`soc_latency`	E4	SYS-08, DMA-11, ACC-20
Regressão e estrutura			
T25	`regress_unit` (`make check` + T02/T03 por seed)	—	O1
T26	`regress_subsys` (T10–T14 × seeds × 3 N, merge no IMC; tag `e3`)	—	O5, O8
T27	`regress_soc` (T16–T24 × ≥ 10 seeds × 3 N, IMC, vereditos; tag `e4`)	—	SYS-05, UVM-04, O8
T28	`lint_elab_3cfg` (Verible + elaboração nas 3 configurações + elaboração negativa)	—	ACC-18, SYS-09
T29	`demo_waveform` (operação completa no SimVision: configuração pela CPU, leituras do DMA, processamento no array, escrita de C, término)	E4	roteiro da demonstração

3 Cronograma preliminar

Cinco semanas a partir de 16/11/2026, alinhadas às entregas do projeto: E3 em 22/11, E4 em 13/12 e apresentação em 18–19/12. A ordem acompanha o estado do RTL (Tabela 3): acelerador e infraestrutura primeiro, porque já existem; DMA, CSR, memória e CPU conforme forem entregues. Os marcos M1 a M3 coincidem com as tags `e3`, `e4` e `final`. Datas a ajustar com o orientador.

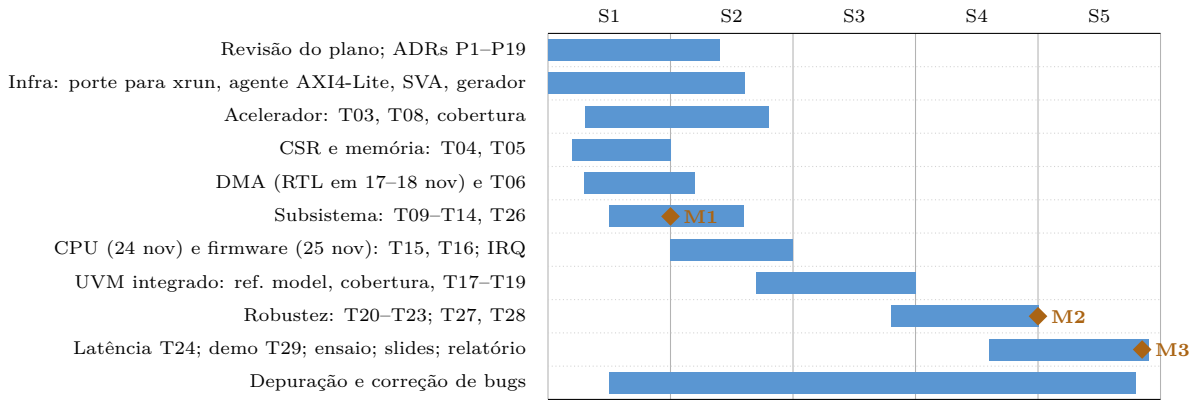


Figura 3: Cronograma preliminar (S1 = semana de 16/11/2026). M1 = tag `e3` (22/11); M2 = tag `e4` (13/12); M3 = apresentação e demonstração (18–19/12).

Tabela 9: Etapas, datas e entregáveis.

Semana	Período	Atividades e entregáveis
S1	16/11 – 22/11	Revisão do plano com a equipe; ADRs dos pontos bloqueantes (P1, P8, P9, P10, P12, P13); ICD v1.1 com mapa de memória e registradores do DMA; porte do testbench do acelerador para `xrun` e `make sim-accel`; T03; RTL e testbench do CSR e da memória (T04, T05); RTL do DMA (17–18/11) e T06; ambiente de subsistema com driver A, agente AXI4-Lite e SVA; T09–T11; regressão por seeds $\times$ N. **M1 (tag `e3`):** DMA, memória e acelerador aprovados juntos.
S2	23/11 – 29/11	T12–T14; cobertura `cg_axil`, `cg_accel_*`, `cg_dma_op`; T08; core (24/11) e firmware genérico (25/11) no `soc_top`; T15 e T16 com driver B por *factory override*; decisão sobre interrupção (29/11); plano v0.2.
S3	30/11 – 06/12	UVM integrado completo: reference model por TLM, monitor de barramento, `cg_sys`; T17–T19; regressão nas três configurações; análise dos buracos de cobertura. Relatórios de síntese dos blocos (06/12, fora deste plano).
S4	07/12 – 13/12	Robustez: T20 (se IRQ), T21–T23; T27 e T28; exclusões de cobertura justificadas; README de verificação com os comandos de compilação, simulação e regressão. **M2 (tag `e4`):** regressão final com vereditos e cobertura no IMC.
S5	14/12 – 19/12	T24 (latência para o comparativo de desempenho); T29 ensaiado no servidor da disciplina, com gravação de contingência; seção de verificação do relatório; slides; tag `final`. **M3:** apresentação e demonstração ao vivo (18–19/12).

Riscos ao cronograma.

(i) **RTL ainda não escrito** (DMA, CSR, memória, core, `soc_top`). Mitigação: modelos comportamentais e driver A permitem fechar o ambiente antes; cada teste entra quando o RTL chega.

(ii) **ICD incompleto** (P9, P10) bloqueia firmware, decodificador e ambiente. Mitigação: ADRs na S1; header do firmware gerado de uma fonte única.

(iii) **Decisão de tiling** (P1, P2, P12) muda a FSM do DMA e o scoreboard. Mitigação: decidir na S1; scoreboard parametrizado pelo *wrap*.

(iv) **Toolchain e licenças**: testbench em Vivado, Xcelium e IMC licenciados, ausência de toolchain RISC-V no servidor. Mitigação: `env_smoke` já prova `xrun` + UVM; `.hex` versionado; porte do testbench na S1.

(v) **Tempo de simulação** com $N = 8$ e vários tiles no nível de sistema. Mitigação: limitar dimensões no aleatório de sistema; dimensões grandes só no subsistema; regressão noturna.

(vi) **Core RV32I**: se próprio e atrasado, a demonstração do top-level fica sem firmware. Mitigação: ADR com core de terceiros como alternativa; o driver A mantém o restante verificado.

(vii) **Demonstração ao vivo**: falha de servidor ou de licença. Mitigação: ensaio no mesmo servidor e com o mesmo usuário; gravação de tela como contingência.

Referências

[1] A. Piziali. *Functional Verification Coverage Measurement and Analysis*. Springer, 2004.

[2] J. Bergeron, E. Cerny, A. Hunter, A. Nightingale. *Verification Methodology Manual for SystemVerilog*. Springer, 2005.

[3] Accellera. *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015; IEEE Std 1800.2-2020.

[4] IEEE Std 1800-2017. *SystemVerilog — Unified Hardware Design, Specification, and Verification Language*.

[5] Arm. *AMBA AXI and ACE Protocol Specification* (AXI4-Lite).

[6] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*.

[7] H. T. Kung. Why systolic architectures? *IEEE Computer*, 15(1), 1982.

[8] CEPEDI / CI Digital. *RISC-V com Coprocessador de Multiplicação Matricial e Controlador DMA* (enunciado do projeto final) e *SD242 — Ambiente UVM para o top-level RISC-V + DMA + Systolic Array* (trabalho final de verificação).

[9] Repositório `riscv-dma-matrix-coprocessor`: `README.md`, `CONTRIBUTING.md`, `02.Architecture/roadmap.md`, `02.Architecture/icd/csr_readme.md`, `04.RTL/accel/readme.md`.